Perfume Collection Analysis

Julissa Dueñas

Goal

As my perfume collection continued to grow, I wanted to take a more intentional, data-driven approach to curating it.

The goal of this project is to streamline my collection, identify a small set of signature scents for each season, and uncover the fragrance notes I naturally gravitate toward based on my wear history.

Data

The two data sets I used for this project are `Master_List.csv` and `Daily_Log.csv`. `Master_List` contains my whole perfume collection. Each perfume has a unique ID, the brand of the fragrance, the name, the type (eau de toilette, body spray, etc.), all of its notes, which size I own, and the price. `Daily_Log` is my wear history: each row records a date from August 28, 2025 and the ID of the fragrance I wore that day (matching `Master_List`).

Data preparation

Using the date from 'Daily_Log' I derived what season it was. I then split each comma-separated fragrance note into seprate rows and assigned each one its own unique ID so wears can be tied to individual notes

```
daily <- daily %>%
  mutate(date = as.Date(date))
daily <- daily %>%
  mutate(date = as.Date(date),
         season = case_when(
           month(date) %in% c(12, 1, 2) ~ "Winter",
           month(date) %in% c(3, 4, 5) ~ "Spring",
           month(date) %in% c(6, 7, 8) ~ "Summer",
           month(date) %in% c(9, 10, 11) ~ "Fall"
         ))
#Split notes
notes_split <- master %>%
  select(id, notes) %>%
  separate_rows(notes, sep = ",") %>%
  mutate(notes = str_trim(notes),
         notes = tolower(notes)) %>%
  distinct()

notes_table <- notes_split %>%
  distinct(notes) %>%
  mutate(note_id = row_number()) %>%
  rename(note_name = notes)
```

```r
perfume_notes <- notes_split %>%
  left_join(notes_table, by = c("notes" = "note_name")) %>%
  select(id, note_id)
```

Database Setup

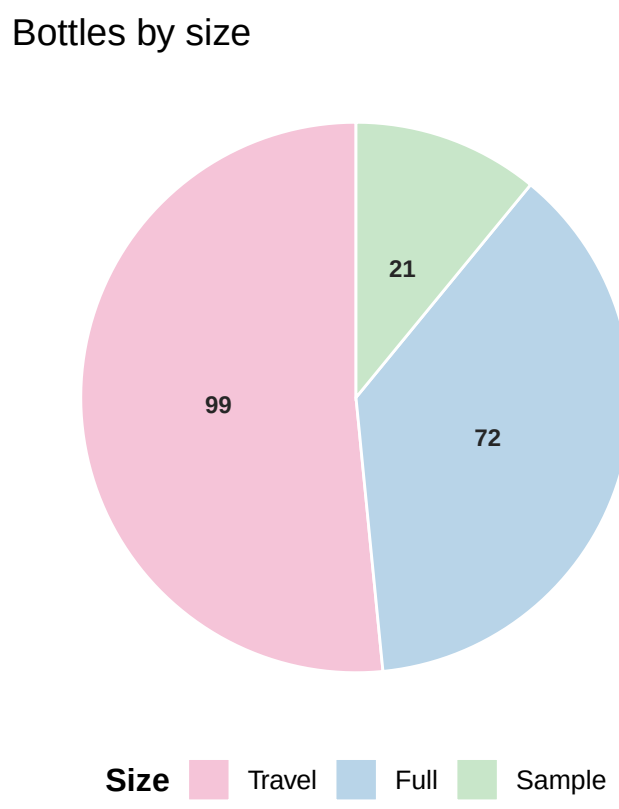
I built a local SQLite database to store the cleaned data and answer my burning questions with SQL. 'perfumes' hold my collection, 'daily_wear_log' holds wears by date and season, Fragrance notes are normalizes into 'note' and 'perfume_notes' provides a bridge so one fragrance can map to multiple ntoes.

```r
con <- dbConnect(RSQLite::SQLite(), "perfume.sqlite")

dbExecute(con, "
CREATE TABLE perfumes (
  id TEXT PRIMARY KEY,
  brand TEXT,
  name TEXT,
  fragrance_type TEXT,
  notes TEXT,
  size TEXT,
  price REAL
);
")
dbExecute(con, "
CREATE TABLE notes (
  note_id INTEGER PRIMARY KEY,
  note_name TEXT
);
")
dbExecute(con, "
CREATE TABLE perfume_notes (
  id TEXT,
  note_id INTEGER
);
")
dbExecute(con, "
CREATE TABLE daily_wear_log (
  date TEXT,
  id TEXT,
  season TEXT
);
")
dbWriteTable(con, "perfumes", master, append = TRUE)
dbWriteTable(con, "notes", notes_table, append = TRUE)
dbWriteTable(con, "perfume_notes", perfume_notes, append = TRUE)
dbWriteTable(con, "daily_wear_log", daily, append = TRUE)
```

Analysis: Now the fun part!

How big is my collection? How many full size perfumes do I have versus travel and sample sizes?

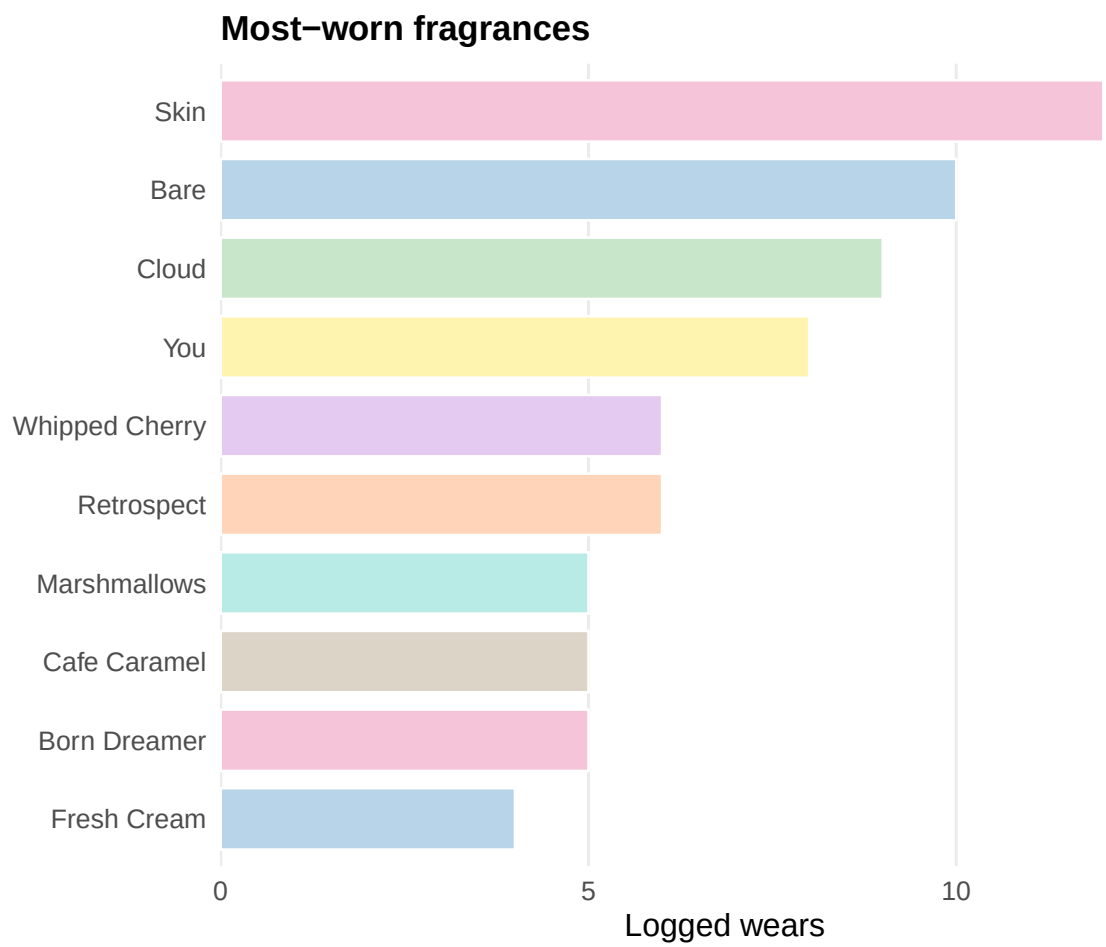
```
size_dist_raw <- dbGetQuery(con, "
SELECT size, COUNT(*) AS count
FROM perfumes
GROUP BY size
ORDER BY count DESC
")

total_perfumes <- sum(size_dist_raw$count)

size_dist <- size_dist_raw %>%
  mutate(
    total = total_perfumes,
    share_pct = round(100 * count / total, 1)
  )
```

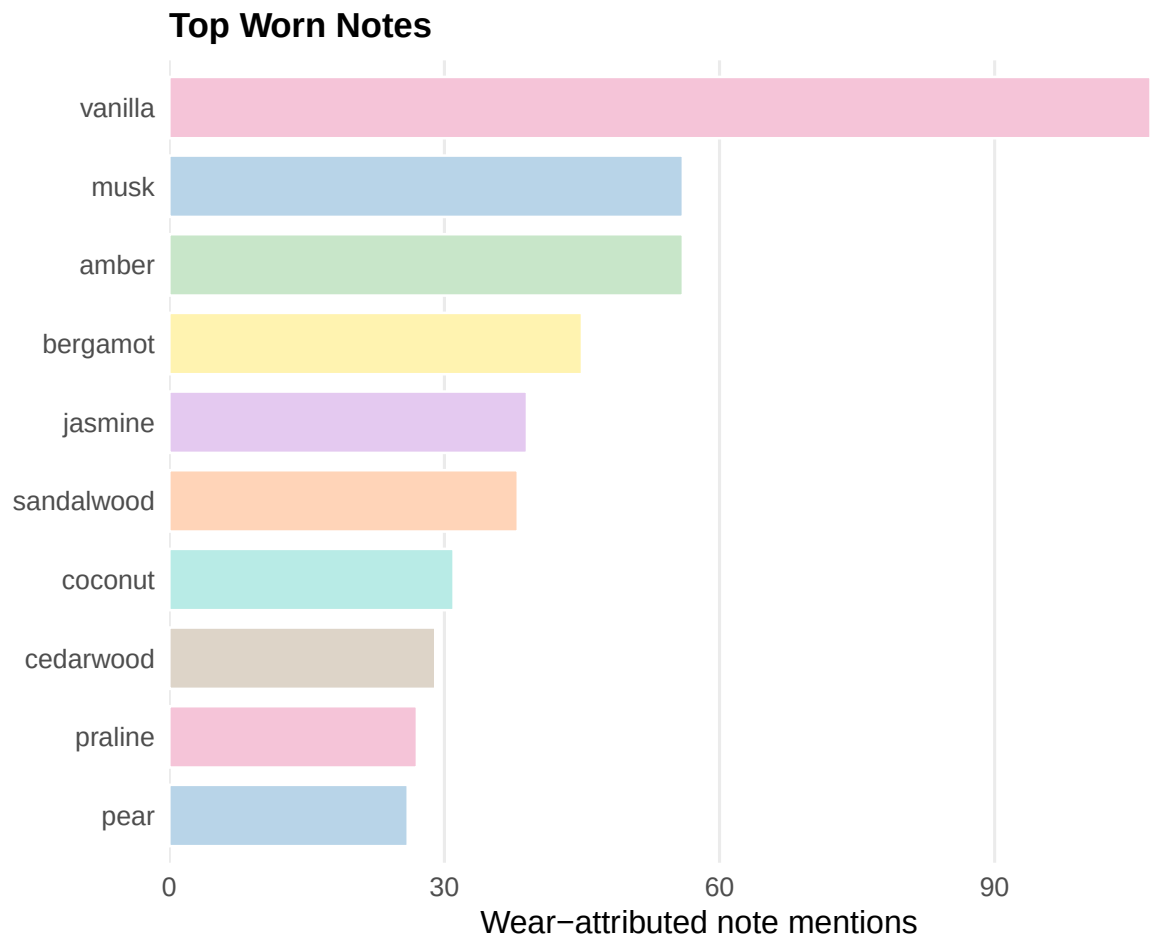


In total, I have 192 perfume bottles (yikes). More than 51% of the collection is travel sized bottles.

What are my top 10 perfumes and the notes I am most reaching for?

```
top10 <- dbGetQuery(con, "
SELECT p.name, COUNT(*) as wears
FROM daily_wear_log d
JOIN perfumes p ON d.id = p.id
GROUP BY p.name
ORDER BY wears DESC
LIMIT 10
")
top_notes <- dbGetQuery(con, "
SELECT n.note_name, COUNT(*) as wears
FROM daily_wear_log d
JOIN perfume_notes pn ON d.id = pn.id
JOIN notes n ON pn.note_id = n.note_id
GROUP BY n.note_name
ORDER BY wears DESC
LIMIT 10
")
```



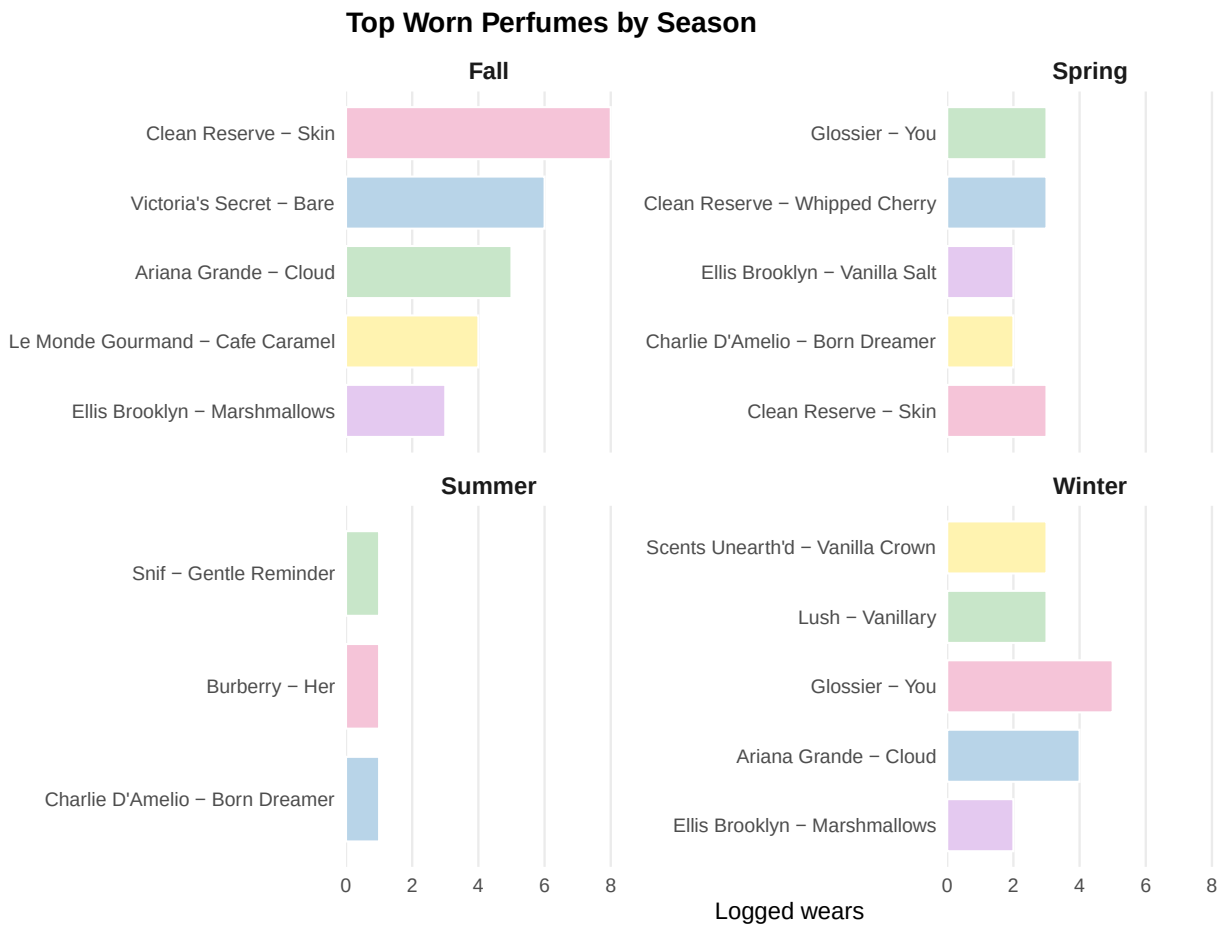


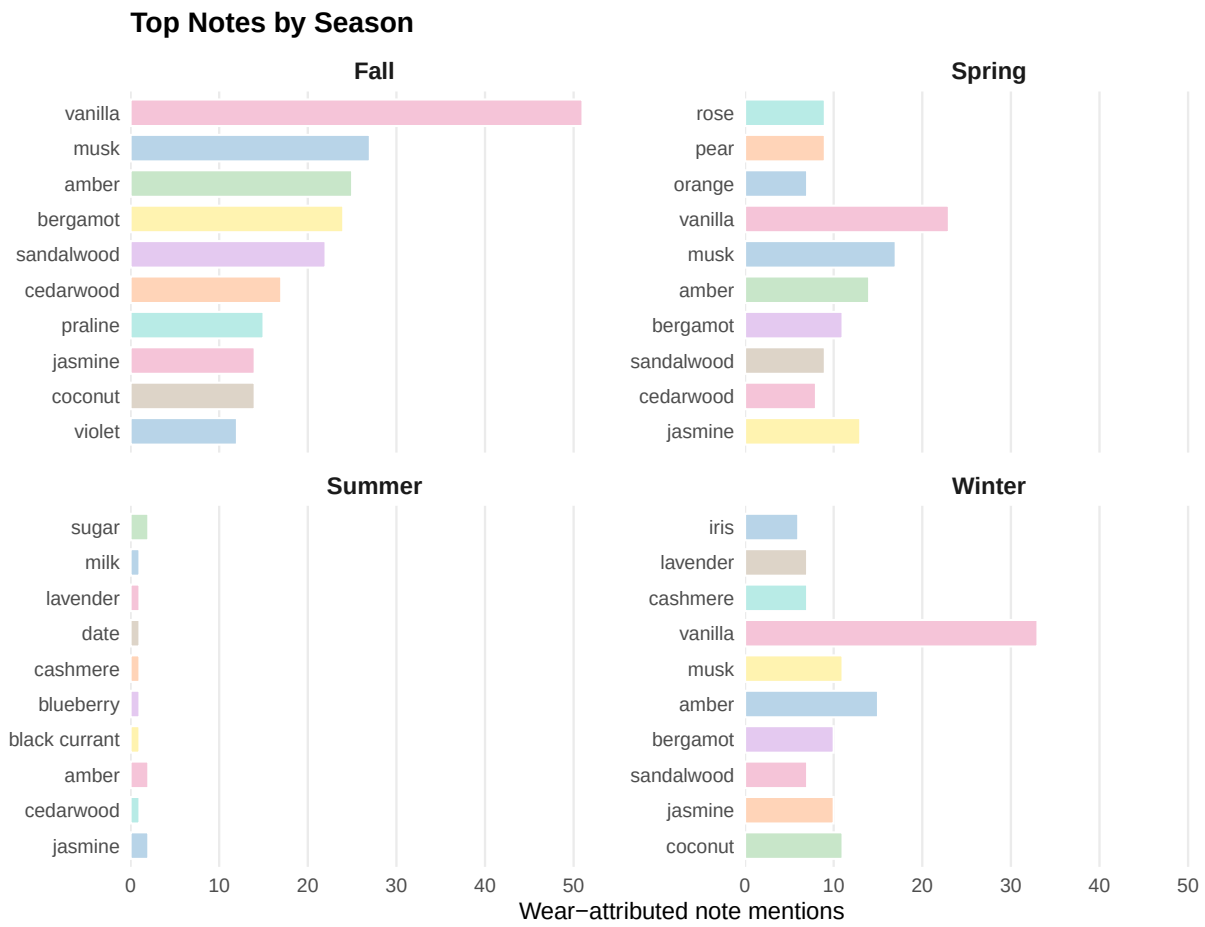
Being as I have 164 days of data, and I have worn a fragrance containing vanilla 107 times, I think it is more than fair to say vanilla earns the top spot

What does this look like by season?

```
top_by_season<-dbGetQuery(con, "
WITH ranked_perfumes AS (
  SELECT
    d.season,
    p.brand,
    p.name,
    COUNT(*) AS wear_count
  FROM daily_wear_log d
  JOIN perfumes p ON d.id = p.id
  GROUP BY d.season, p.brand, p.name
)
SELECT *
FROM (
  SELECT *,
         ROW_NUMBER() OVER (
           PARTITION BY season
```

```
          ORDER BY wear_count DESC
        ) AS rank
  FROM ranked_perfumes
)
WHERE rank <= 5
ORDER BY season, rank;
")
top_notes_by_season <- dbGetQuery(con, "
WITH note_counts AS (
  SELECT
    d.season,
    n.note_name,
    COUNT(*) AS note_count
  FROM daily_wear_log d
  JOIN perfume_notes pn ON d.id = pn.id
  JOIN notes n ON pn.note_id = n.note_id
  GROUP BY d.season, n.note_name
)
SELECT *
FROM (
  SELECT *,
        ROW_NUMBER() OVER (
          PARTITION BY season
          ORDER BY note_count DESC
        ) AS rank
  FROM note_counts
)
WHERE rank <= 10
ORDER BY season, rank;
")
```


Top Worn Perfumes by Season
Fall
Clean Reserve − Skin
Victoria's Secret − Bare
Ariana Grande − Cloud
Le Monde Gourmand − Cafe Caramel
Ellis Brooklyn − Marshmallows
Spring
Glossier − You
Clean Reserve − Whipped Cherry
Ellis Brooklyn − Vanilla Salt
Charlie D'Amelio − Born Dreamer
Clean Reserve − Skin
Summer
Snif − Gentle Reminder
Burberry − Her
Charlie D'Amelio − Born Dreamer
Winter
Scents Unearth'd − Vanilla Crown
Lush − Vanillary
Glossier − You
Ariana Grande − Cloud
Ellis Brooklyn − Marshmallows
0
2
4
6
8
Logged wears



I don't have too much data from Summer months just yet, but already, I see some floral and fruity notes coming in for the Spring months. I am curious to see how the notes differ based on the seasons.